universität freiburg

Hahn Schickard

Intelligent Embedded Systems Lab

Investigating Sparse Representations and Deep Metric Learning for Clustering of Biological Nanopore Current Traces of Peptides

By: Rushang Phira

5578541

Supervisor: Dr. Göran Köber

First Examiner: Prof. Dr. Oliver Amft

Second Examiner: Prof. Dr. Rolf Backofen

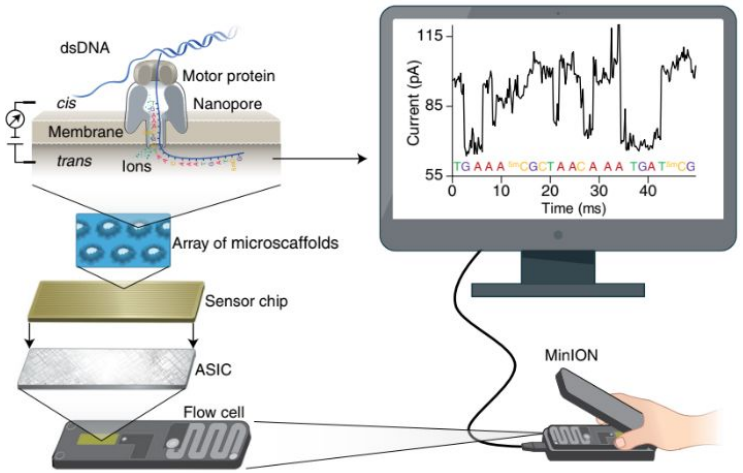
An Introduction to Nanopore Sensing

Principle:

- Molecule enters pore: blocks ionic current
- Creates a transient current drop (blockade event)
- Blockade features like:
    - Depth (I/I0)
    - Duration
    - Shape & fluctuations

Challenges

- Data is noisy & high-dimensional
- Subtle differences in PTMs are hard to detect
- We need to extract meaningful patterns in absence of labels

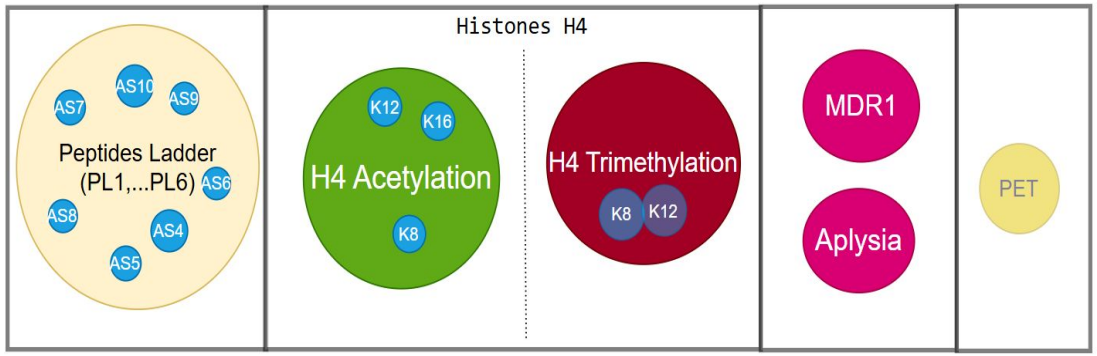


MinION flow cell [1]

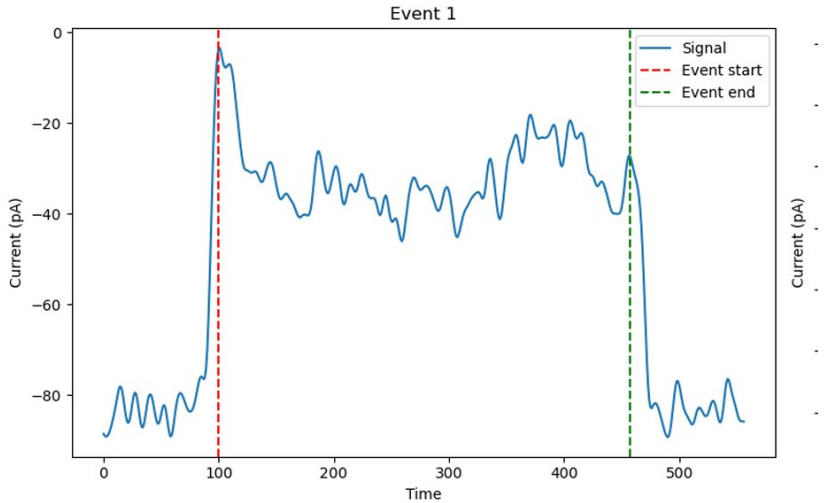
Thesis Goals

- Addressing unsupervised learning to cluster nanopore signals (Previous literature focuses on classification)
- **Problem**: Analyzing PTMs (post-translational modifications) remains challenging, with a research gap in clustering methods.
  - Most methods rely heavily on <u>mean-related features</u>
- **Goal**: Develop an adaptive clustering framework that works without mean-related features to capture PTM-specific patterns.
- **Contributions:**
  - *Sparse K-means* for inter-analyte clustering
  - *Triplet-based deep clustering* for intra-analyte PTM separation

Datasets and Events

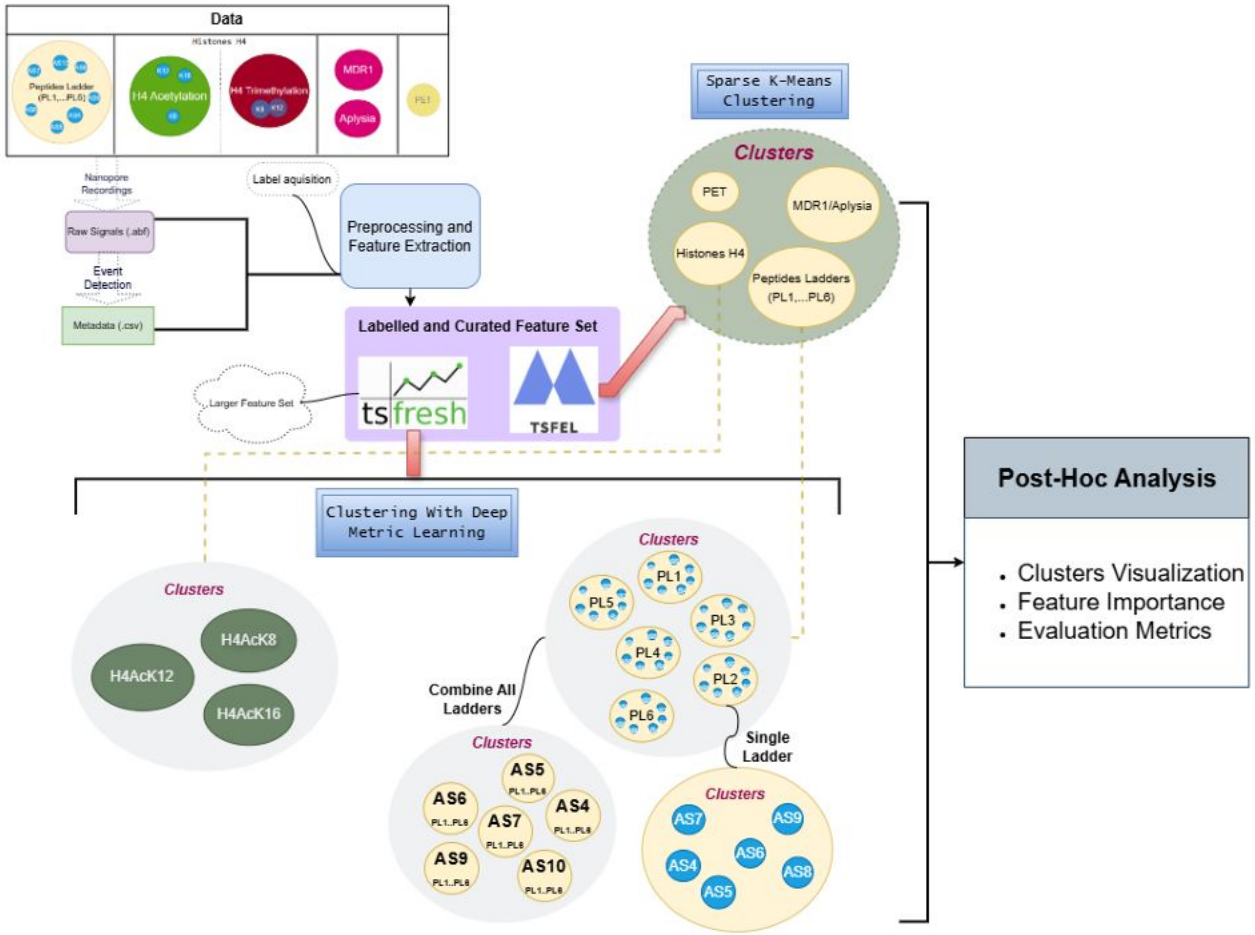


Our dataset contains 5 labels of analytes at the broadest level, with histones containing several post-translational modifications (PTMs)

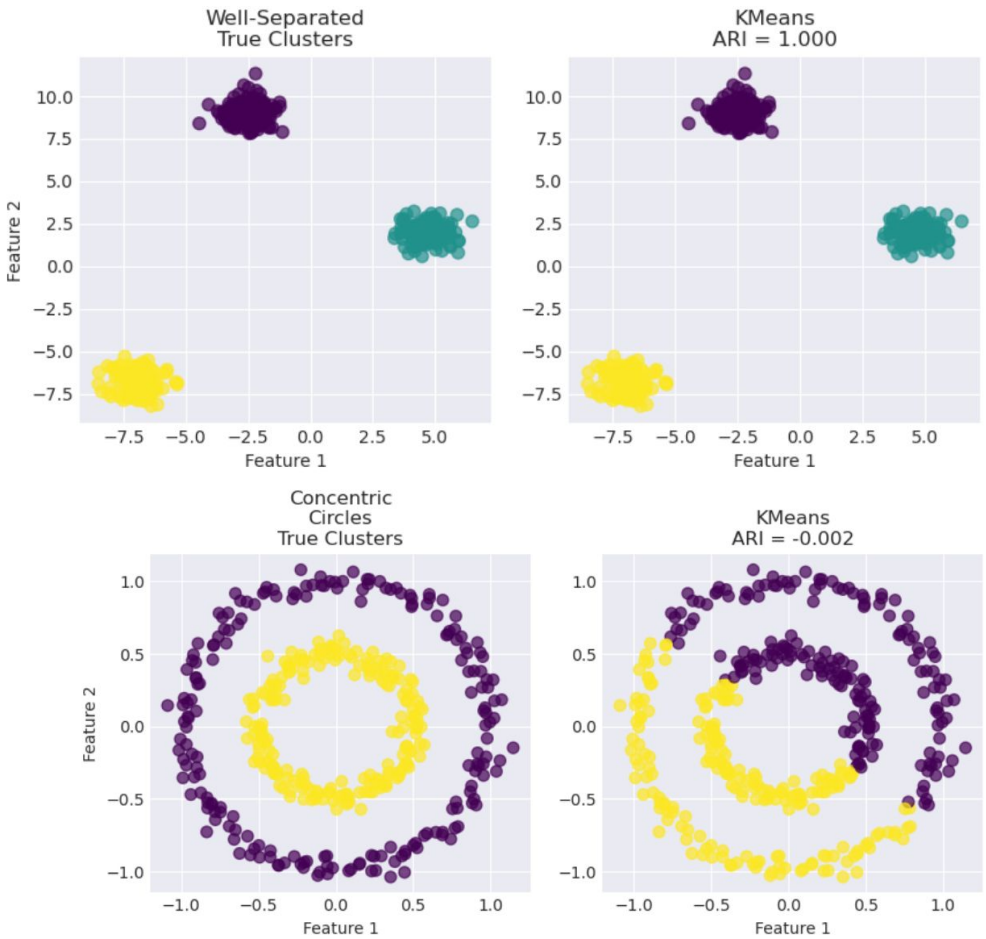


A sampled unlabeled blockade from the PL2 dataset containing PTMs AS4-AS10. We see the deviations from the baseline of -80pA as the analyte passes into the event state marked by the sharp spike.

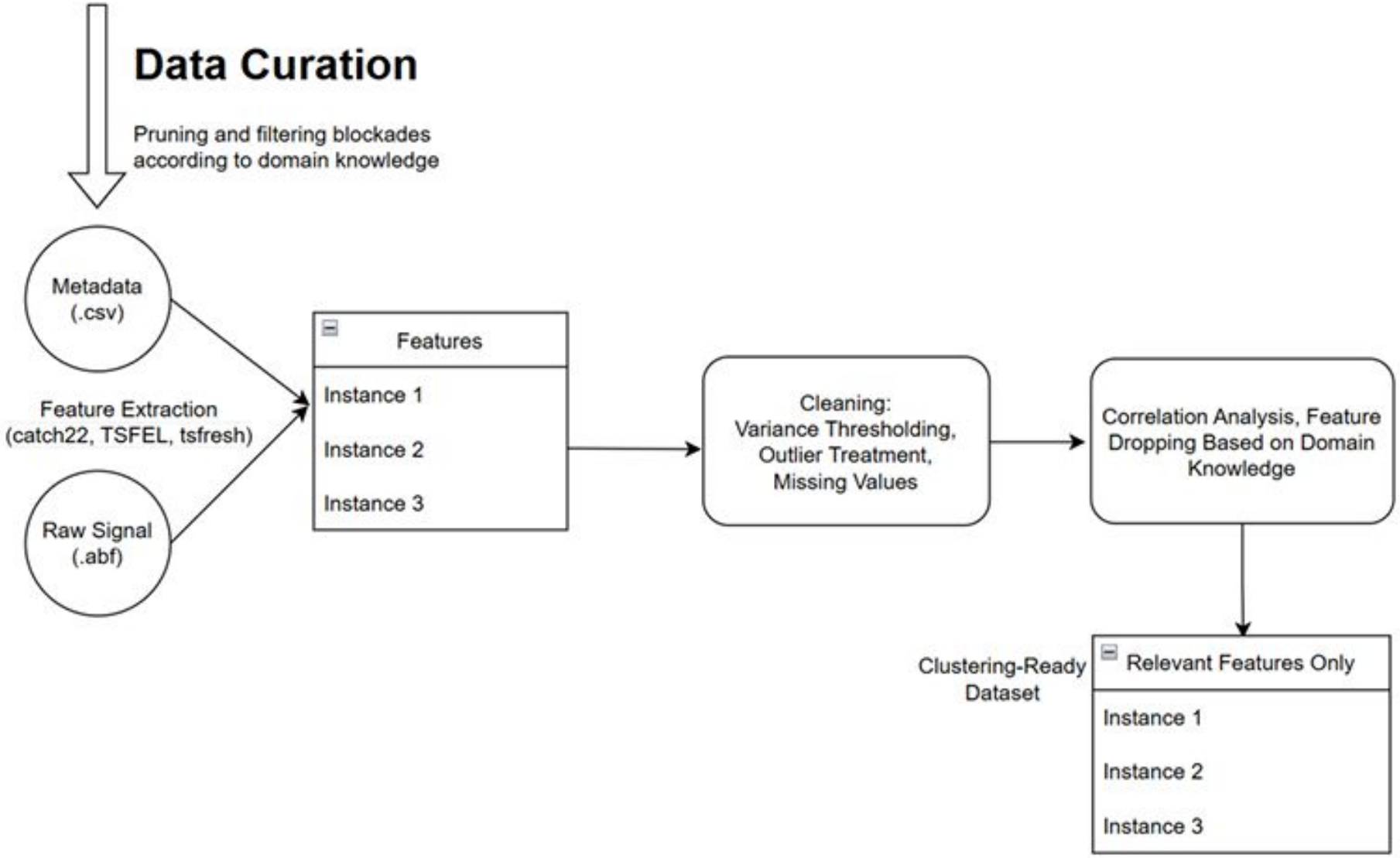
Methodology Overview



Evaluation: ARI, UMAP visualization, feature importance analysis



Preprocessing



Feature Pools

- **CAnonical Time-series CHaracteristics (Catch-22) [*22 Features*] [3]**
  - Used preliminarily but was outperformed by more comprehensive pools
- **Time Series Feature Extraction Library (TSFEL) [*400 Features*] [4]**
  - Works best for inter-analyte clustering
- **Time Series Feature Extraction based on Scalable Hypothesis tests (tsfresh) [*700+ Features*] [5]**
  - Most comprehensive and worked best for PTM (intra-analyte) clustering

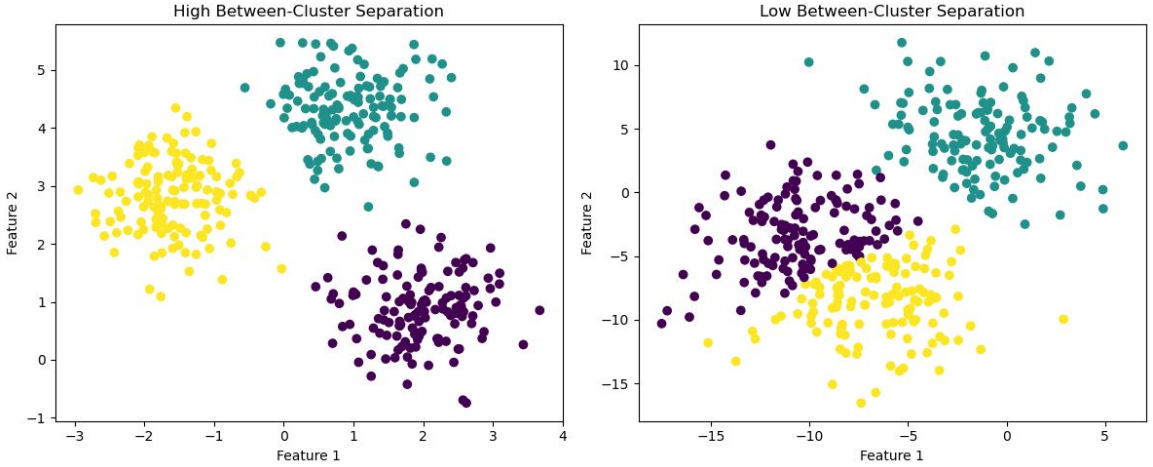
Sparse K-Means [2]

- Based on Witten & Tibshirani (2010) [2]
- Integrates feature selection into clustering via L1 sparsity constraint
- Weights features dynamically; irrelevant features get ~0 weight
- works well for broad analyte separation

Sparse K-Means [2]

- **Step 1**: Weight initialization
- **Step 2**: Cluster with weighted features
- **Step 3**: Update feature weights based on cluster separation
  - compute the Between-Cluster Sum of Squares (BCSS) for each feature.
    - BCSS = Total Sum of Squares (TSS) – Within Cluster Sum of Squares (WCSS)
- **Step 4**: Apply sparsity via soft-thresholding
  - Binary search to find the threshold
- **Step 5**: Normalize and repeat

Two plots showing high and low BCCS clustering respectively



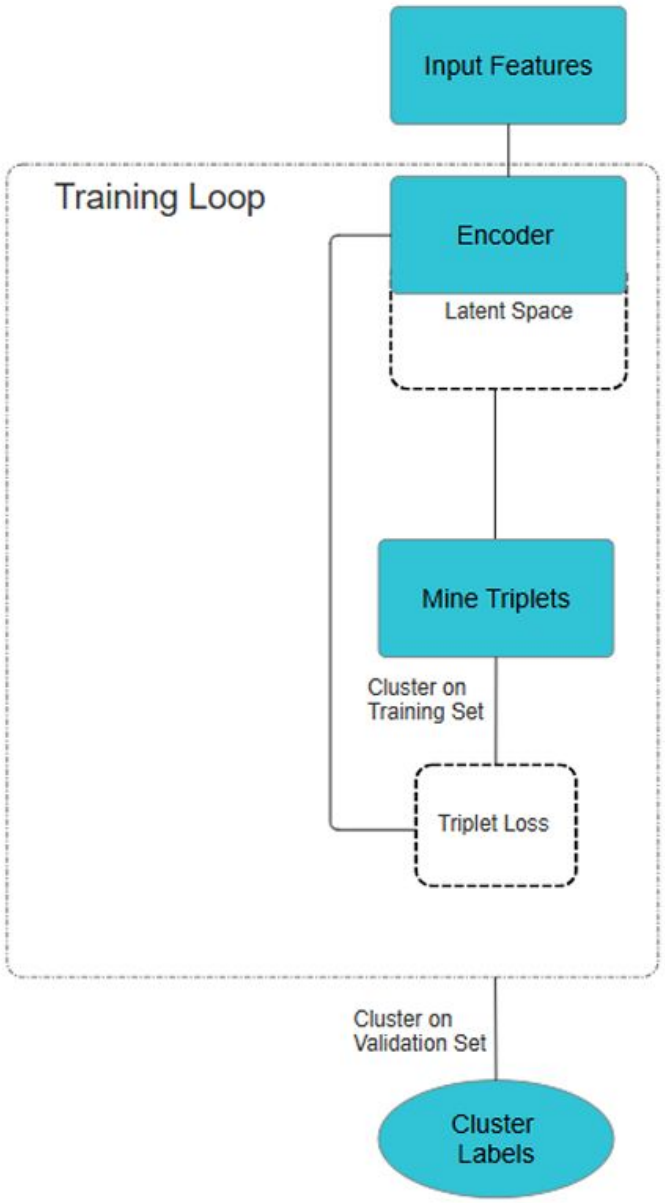
Algorithm 1 Sparse K-Means Clustering algorithm; where $\mathbf{X}$: Data matrix ($n \times p$), where n is the number of samples and p is the number of features.
$\mathbf{w}$: Feature weight vector in sparse K-means.
s: Sparsity parameter controlling the L_1 norm of $\mathbf{w}$.
K: Number of clusters.
C_k: The k-th cluster.
a (BCCS): Between cluster sum of squares
TSS: Total sum of squares
WSS: Within-Cluster sum of squares

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$, K, s, τ
Ensure: Clusters $\{C_k\}$, weights $\mathbf{w}$

```
w ← [1/√p]^p                                   ▷ Initialize weights
repeat
    X_w ← X · diag(√w)
    {C_k} ← KMeans(X_w, K)                     ▷ Cluster with current weights
    for j ← 1 to p do                          ▷ Compute BCSS
        TSS ← Σ_i (X_ij − μ_j)^2
        WSS ← Σ_k Σ_{i∈C_k} (X_ij − μ_kj)^2
        a_j ← TSS − WSS
    end for
    Δ* ← FindDelta(a, s)                       ▷ Binary search for threshold
    w_new ← S(a, Δ*)/‖S(a, Δ*)‖_2
    change ← ‖w_new − w‖_1 / ‖w‖_1
until change < τ
return {C_k}, w_new
```

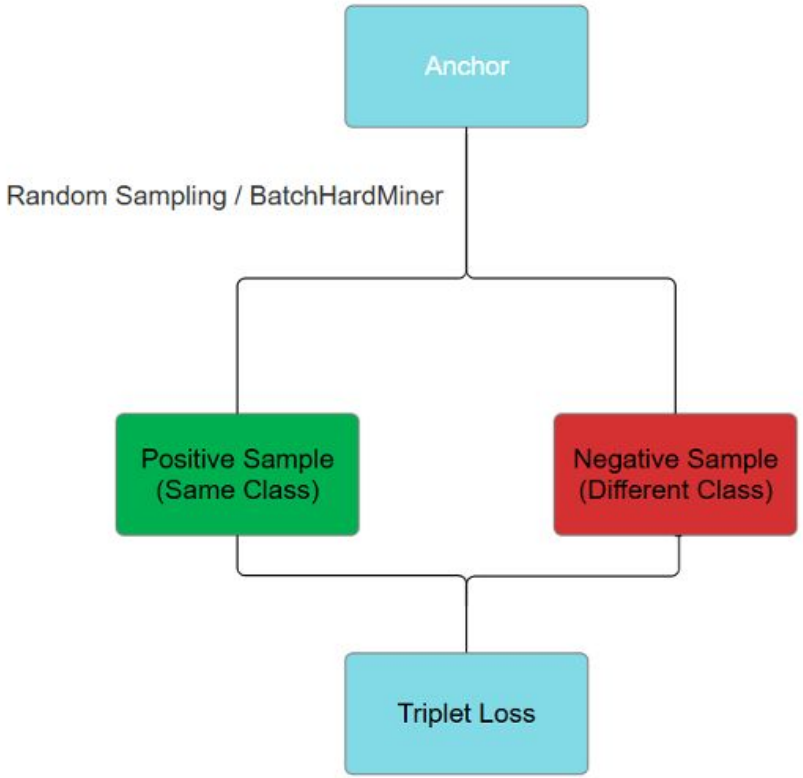
Triplet-Loss-Based Encoder [6]

- Encoder network maps high-dim features to low-dim latent space
  - Deep feedforward network with four hidden layers
- Triplet loss: Anchor, positive (same class), negative (different class)
- Mining strategy: Random triplets mined
- Feature importance: Via Elastic Net regression on latent dimensions (VIF)

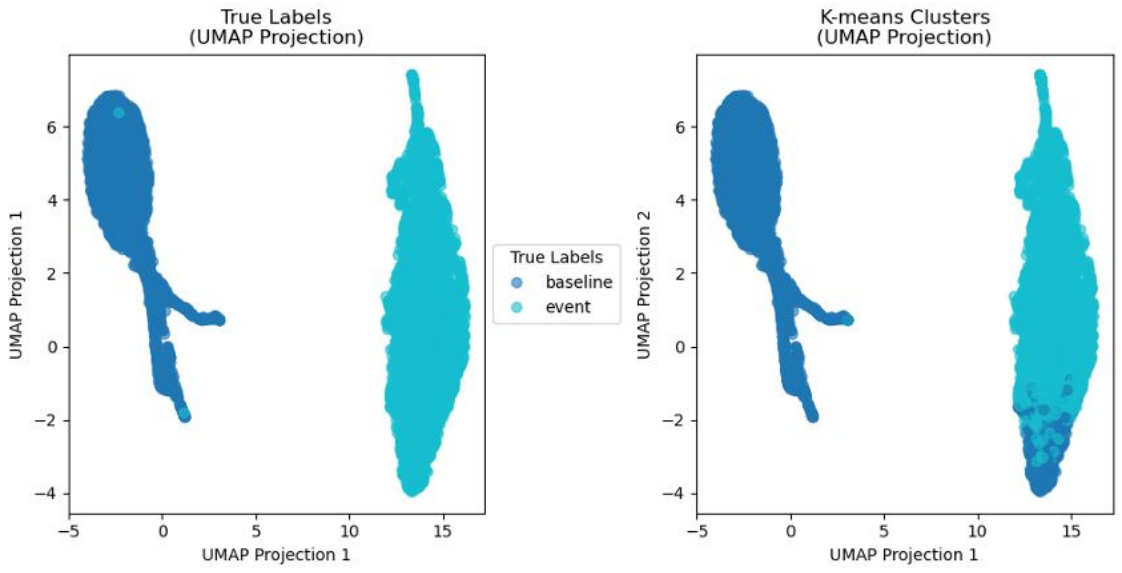


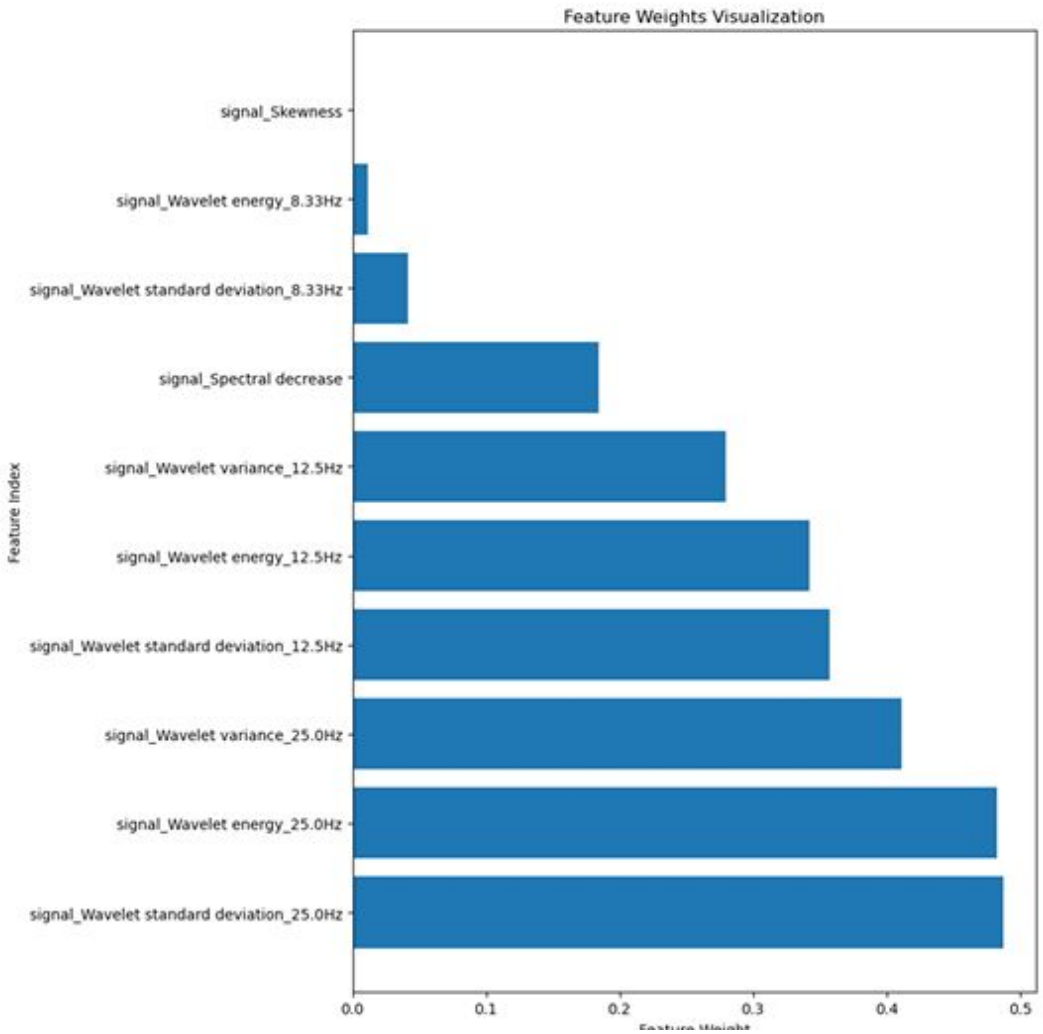
Triplet-Loss-Based Encoder [6]

- Distance(Anchor, Positive) < distance(Anchor, Negative)
- Setting margin to 1



Distinguishing Baseline From Blockade

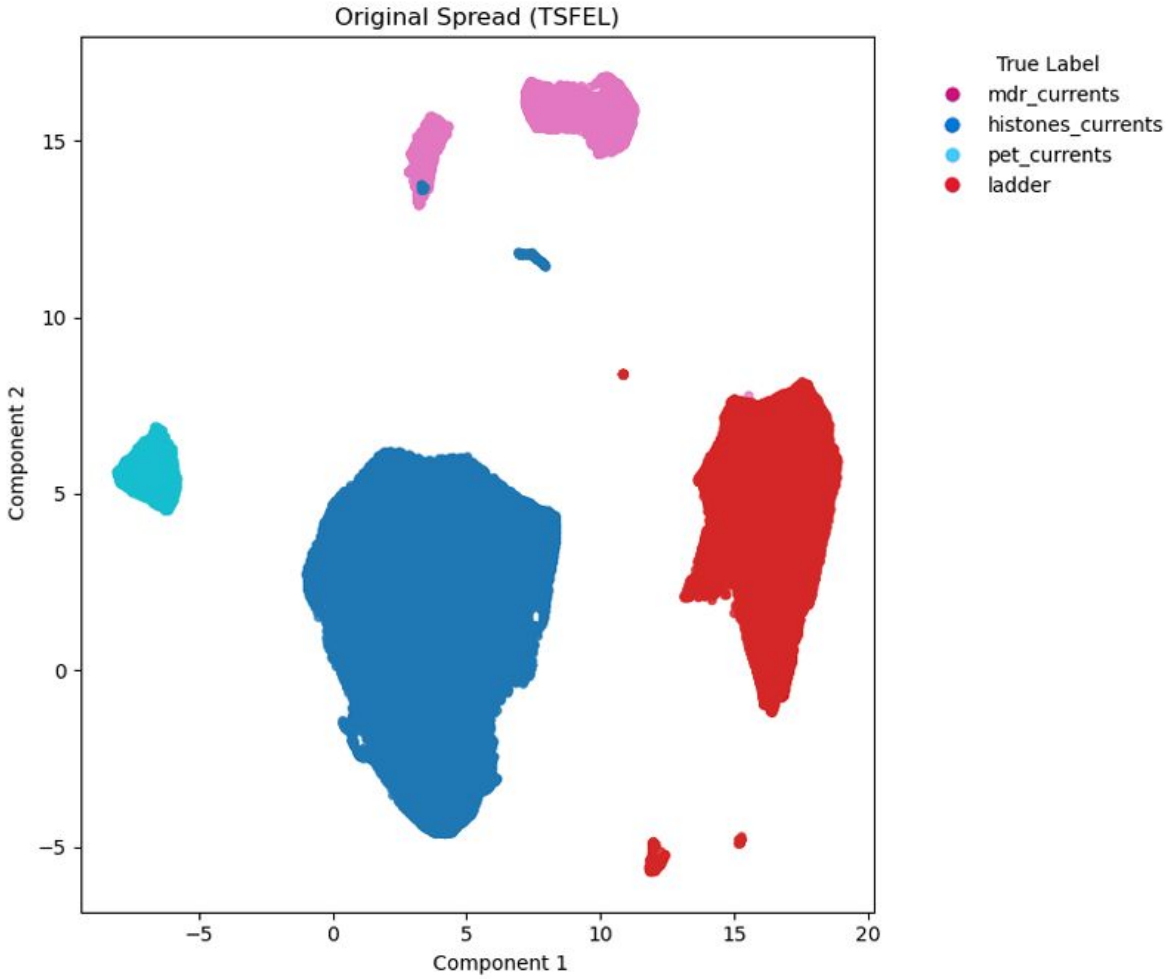


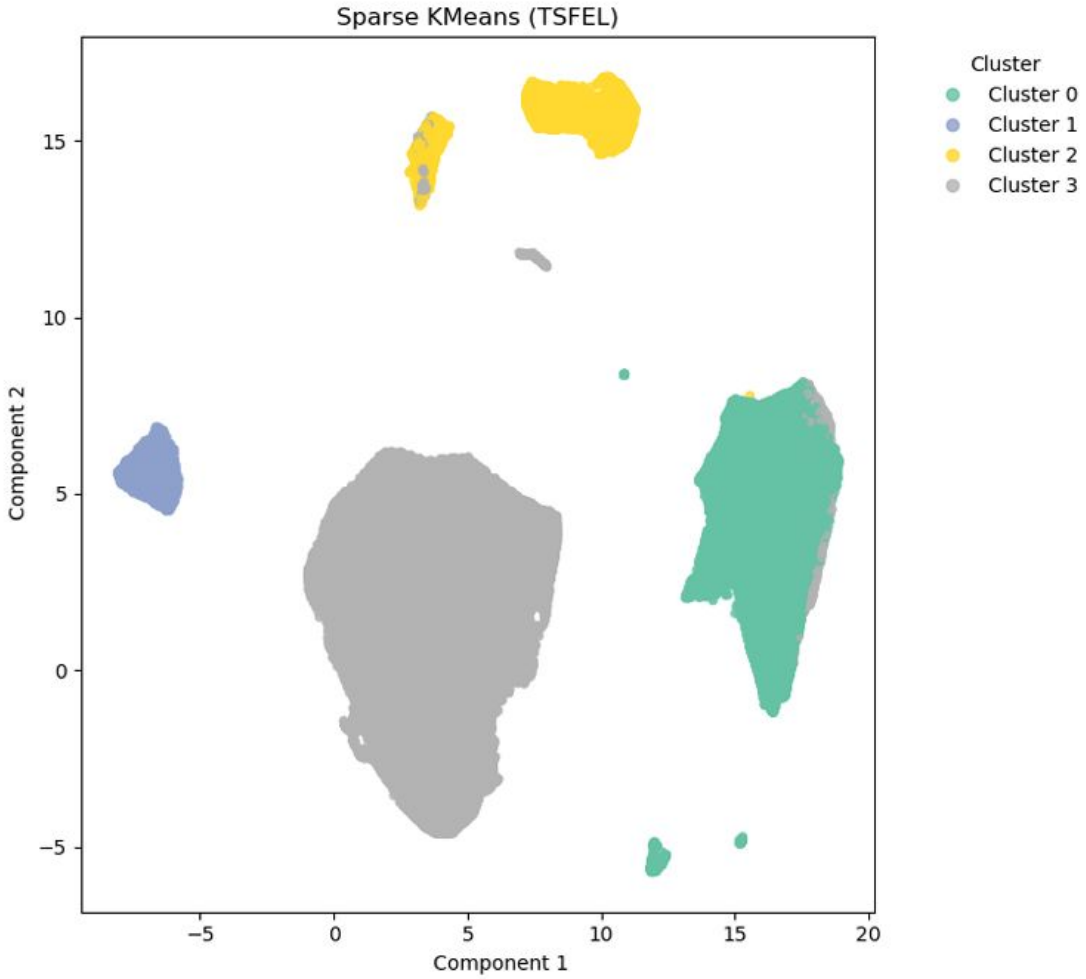


- Sparse K-Means on TSFEL feature pool
- **Adjusted Rand Index (ARI) ≈ 0.76**

Inter-Analyte Clustering

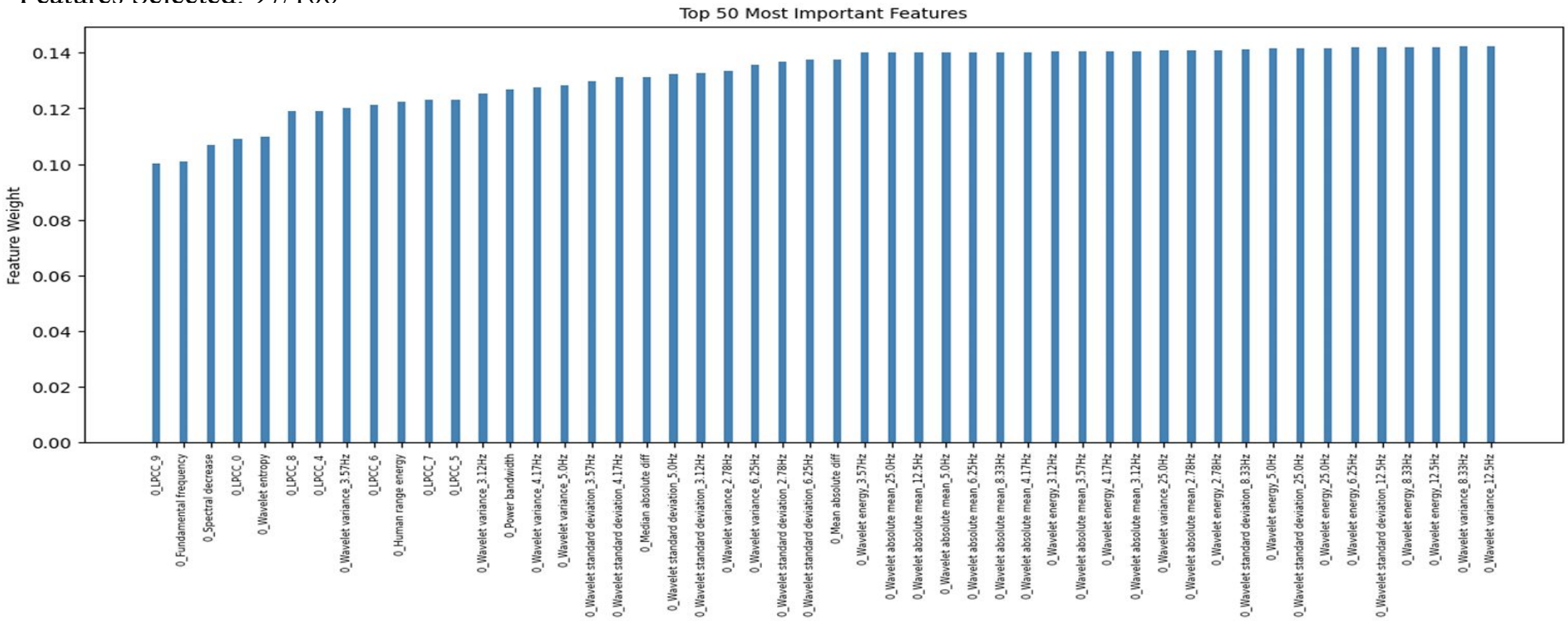
- Improvement in ARI (0.934 vs. 0.948) over regular K-Means
- approximately 0.319% (164 of 145612) samples reassigned correctly



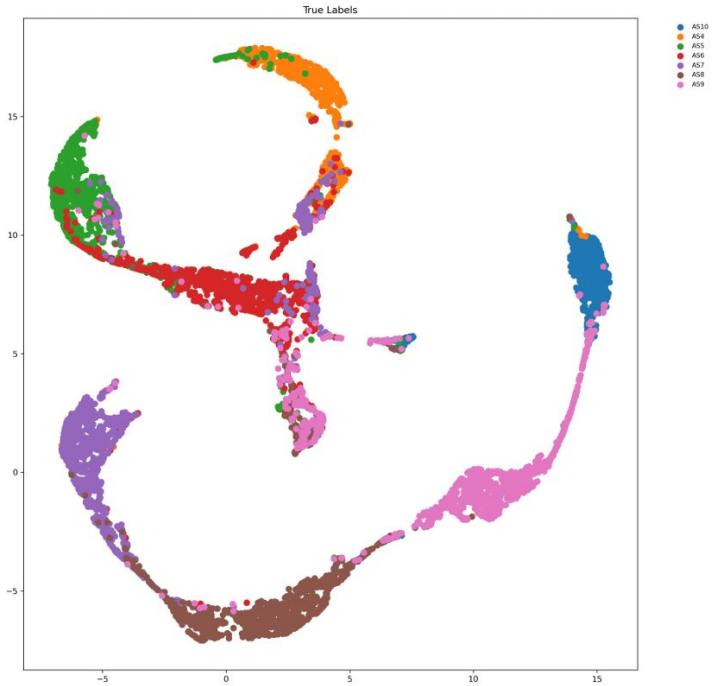


Inter-Analyte Clustering

- Sparsity constraint = 8.6
- Features Selected: 97/106



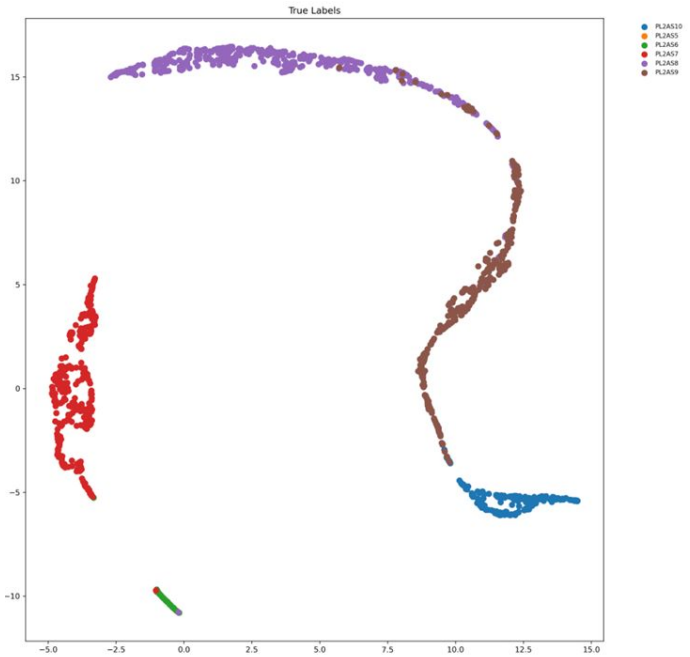
All Peptides Ladders PTMs Clustering



Latent embeddings for a combined dataset of all AS4-AS10 PTMs of peptides ladders

Class	Precision	Recall	F1-score	Support
AS4	0.978	0.989	0.983	615
AS5	0.925	0.938	0.932	670
AS6	0.936	0.908	0.922	545
AS7	0.972	0.981	0.977	802
AS8	0.978	0.987	0.982	1002
AS9	0.996	0.968	0.982	753
AS10	0.974	0.996	0.985	712

Peptides Ladder 2 Clustering



Latent embeddings for Peptides Ladder 2 AS4-AS10

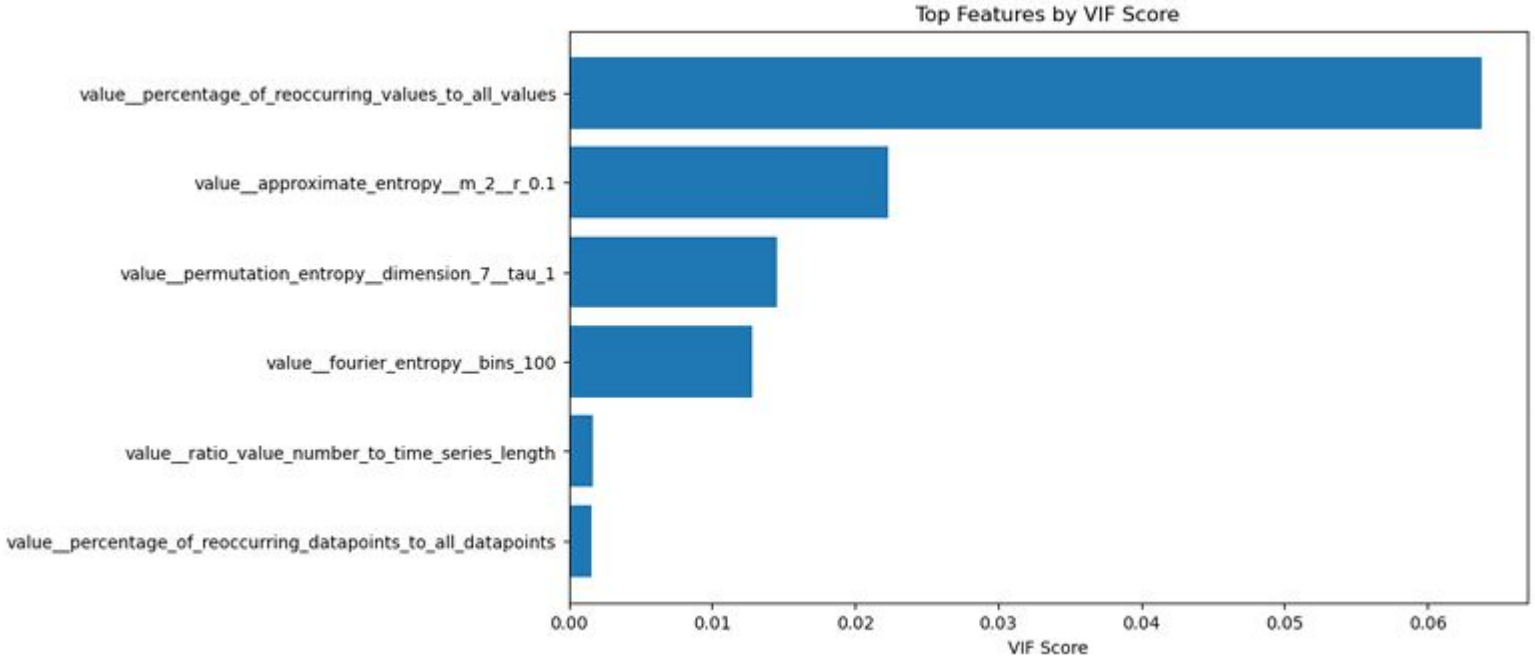
Class	Precision	Recall	F1-score	Support
AS5	0.0000	0.0000	0.0000	41
AS6	0.0000	0.0000	0.0000	147
AS7	0.8306	1.0000	0.9075	932
AS8	1.0000	0.9663	0.9829	832
AS9	0.6379	1.0000	0.7786	327
AS10	0.0000	0.0000	0.0000	161

- Clusters formed are typically high-purity

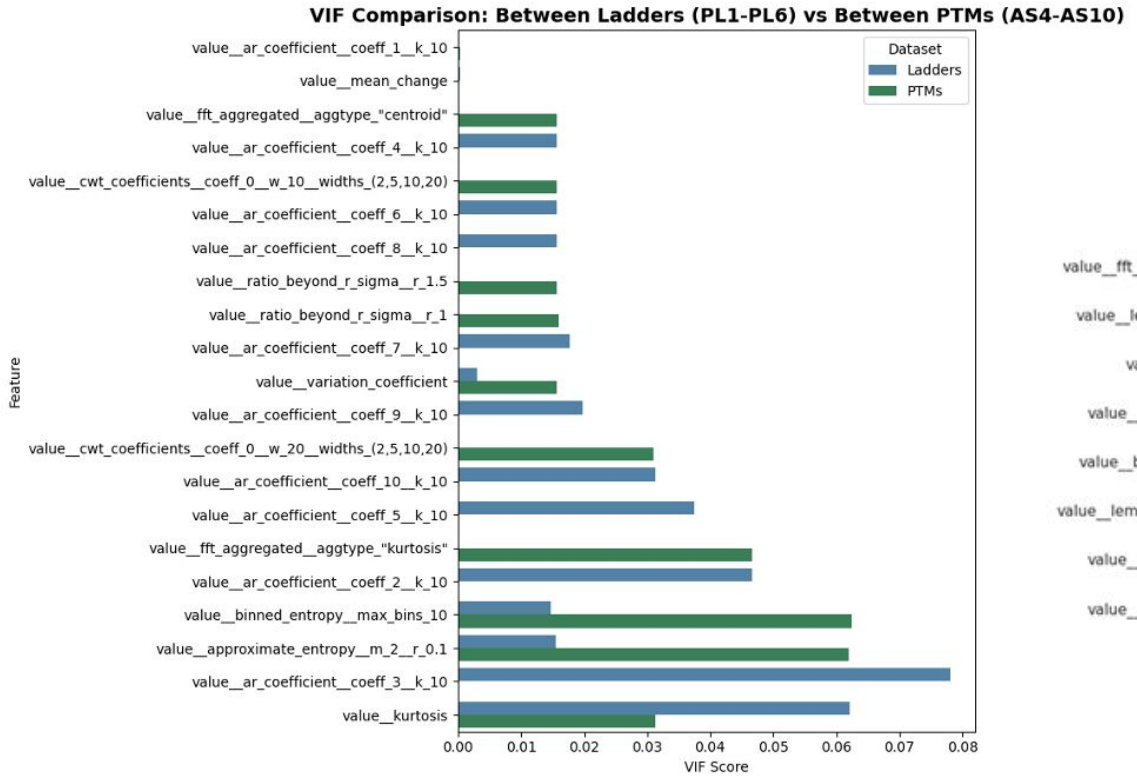
Feature Analysis with Variable Inclusion Frequencies

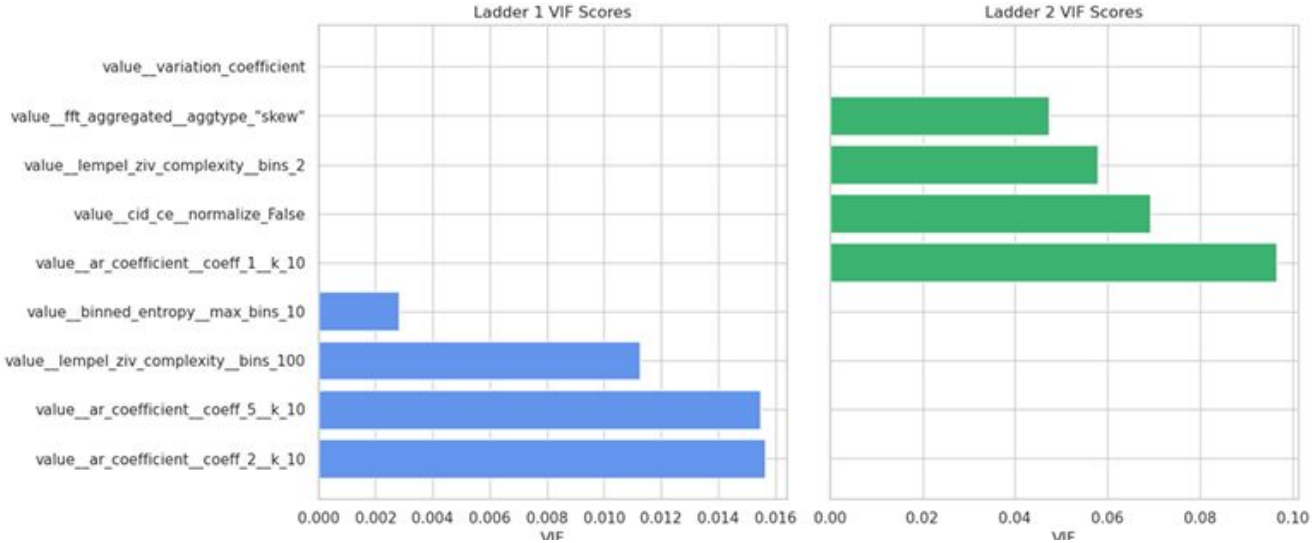
- Deep learning models are often "black boxes"
  - Which features actually contribute to clustering?
  - How do we interpret learned latent spaces?
- VIF with Elastic Net
  1. Take learned latent embeddings
  2. Fit Elastic Net regression to predict each latent dimension
  3. Use Lasso penalty (sparse feature selection)
  4. Repeat across ‘n’ bootstrap samples
  5. Calculate Variable Inclusion Frequency:

Feature Analysis on Histones Acetylation (H4Ac)



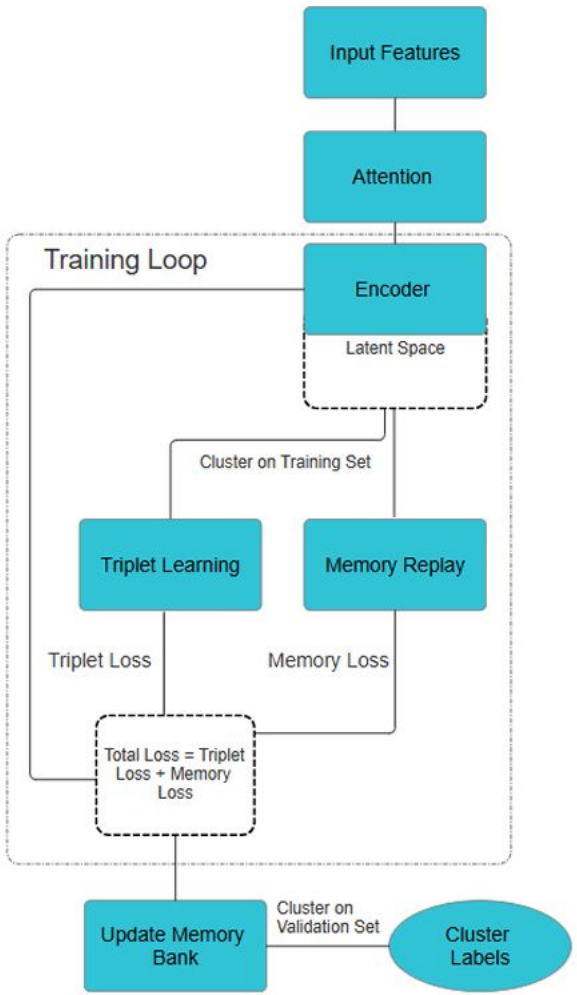
Feature Analysis Summary





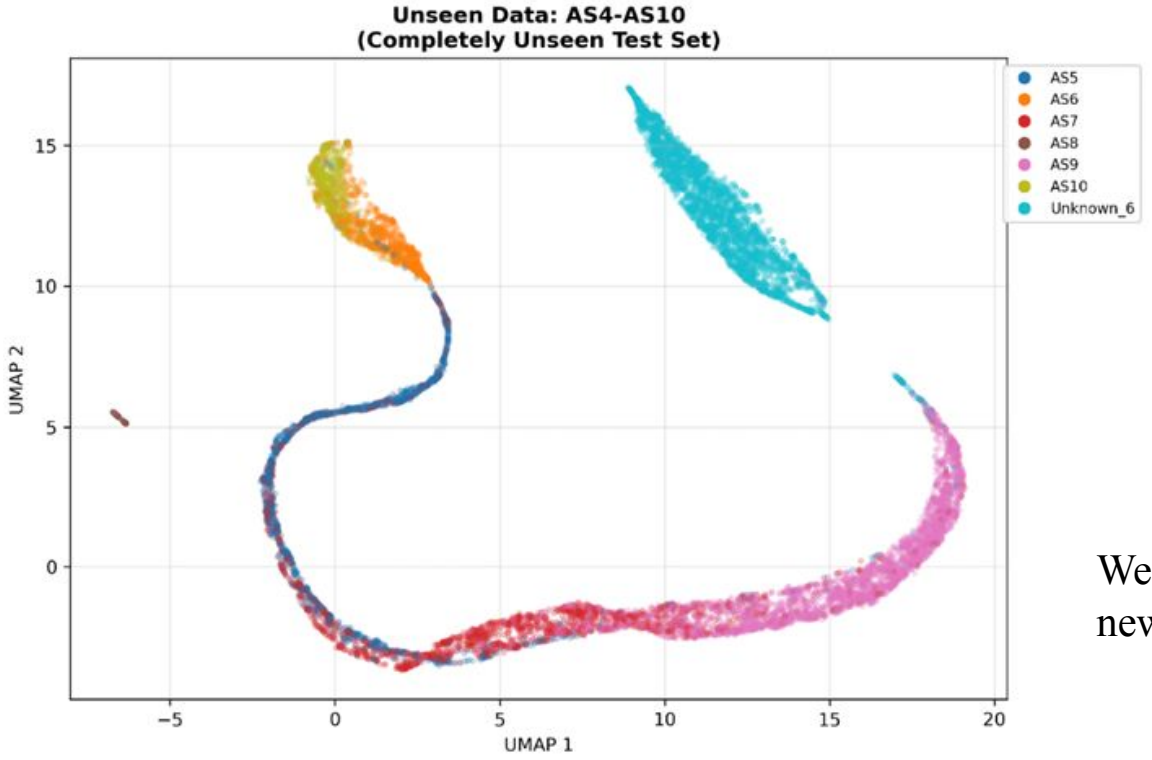
Memory-Attention [7]

- Handles cumulative data without catastrophic forgetting
- Adds attention mechanism (feature weighting) and memory bank (replay old samples)
- Inspired by iCaRL[7] but adapted for clustering



Memory-Attention - Results

	Ladder 2		Ladder 1	
Metric	**Final Step**	**All Steps**	**Final Step**	**All Steps**
ARI	0.7491	0.7520	0.6240	0.3024
NMI	0.7582	0.7710	0.6942	0.4376
Homogeneity	0.9159	0.7014	0.8700	0.5984



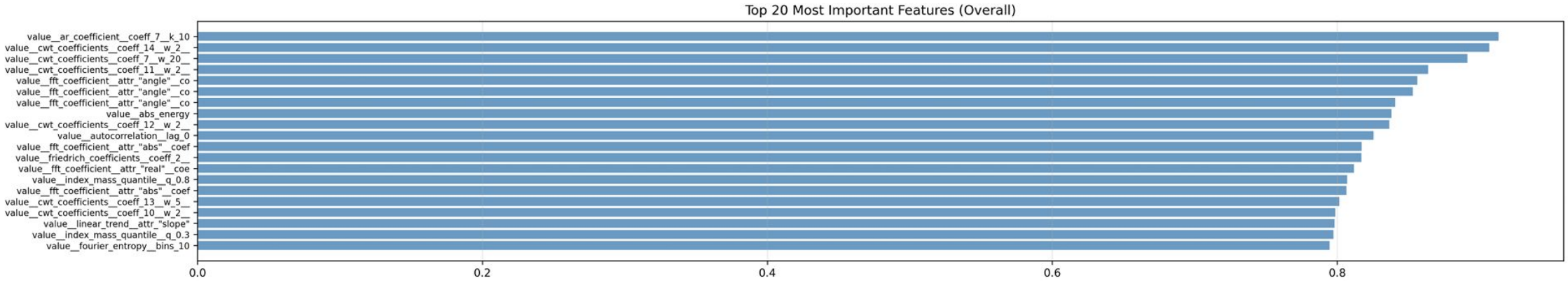

```
UNSEEN TEST RESULTS
==============================
Overall Metrics:
• ARI: 0.5932
• NMI: 0.6464
• Silhouette: 0.5900

Dataset Info:
• Samples: 11478
• Proteins: 7

Per-Protein F1 Scores:
• AS5: 0.649
• AS9: 0.632
• Unknown_6: 0.982
• AS7: 0.499
• AS6: 0.676
• AS10: 0.000
```

We treat the last iteration of PL1 (AS4-AS10) as a new, unseen test set

Memory-Attention – Feature Analysis



Conclusion

- We successfully cluster analytes & PTMs without mean-related features

Future Work:

- Self-supervised approach for triplet-loss
- Raw signal processing (CNNs/Transformers)
- More data per experiment for robust clustering
- Principal Component Regression (PCR), since VIFs are low

References

[1] Wang, Yunhao, et al. "Nanopore sequencing technology, bioinformatics and applications." *Nature biotechnology* 39.11 (2021): 1348-1365.

[2] A D. M. Witten and R. Tibshirani, “A framework for feature selection in clustering,” Journal of the American Statistical Association, vol. 105, no. 490, pp. 713–726,

2010, PMID:20811510.

[3] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, “Catch22: Canonical time-series characteristics: Selected through highly comparative time-series analysis,”

[4] M.Barandasetal.,“Tsfel: Timeseries feature extraction library,” SoftwareX, vol. 11, p. 100456, 2020.

[5] M. Christ, N. Braun, J. Neuffer, and A. W. Kempa-Liehr, “Time series feature extraction on basis of scalable hypothesis tests (tsfresh–a python package),” Neurocomputing, vol. 307, pp. 72–77, 2018.

[6] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2015

[7] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “Icarl: Incremental classifier and representation learning,” in Proceedings of the IEEE conference on Computer Vision and Pattern Recognition, 2017, pp. 2001–2010.

universität freiburg

Thank You